School LLM

Technical White Paper on the AI / RAG Architecture

Author: Gangadhar Reddy

Date: 26 May 2026

Audience: Senior GenAI / RAG reviewers

A code-grounded narrative of the retrieval pipeline, vector-store design, prompt-engineering strategy, multi-provider LLM orchestration, and quality-assurance loop that I designed and implemented.

Author: Gangadhar Reddy
Date: 26 May 2026
Project: School LLM — Retrieval-Augmented Learning Platform

1. Scope

This brief covers the AI components I designed and built: the RAG pipeline, the vector layer, the prompt engineering for Q&A / summary / quiz, the multi-provider LLM strategy that combines Claude with a local Ollama server, and the performance and quality-assurance machinery around it. Each section states the design choice, the rationale, the implementation, and the tradeoff.

2. System Overview

The platform is a FastAPI backend, a Streamlit frontend serving student / teacher / administrator dashboards, MongoDB Atlas for application state, and a persistent ChromaDB store for embeddings. The end-to-end Q&A path is: PDF is chunked and embedded on first access, the question is rewritten into retrieval variants, top-k chunks are reranked with a metadata-aware scorer, a grounding check decides whether to refuse, and the surviving context is passed to Claude (production) or Ollama (development) with intent-conditional and audience-conditional prompt augmentations.

3. RAG Pipeline

3.1 PDF Extraction — Three-Tier with Structural Recovery

PDFs in a school corpus are heterogeneous — born-digital textbooks, phone scans, printable worksheets — so a single library breaks on roughly one document in five. The handler in **backend/pdf_handler.py** layers three strategies:

- PyMuPDF (primary) for born-digital PDFs. I call **page.get_text("dict")** to retain block / line / span structure, then reconstruct lines from spans and use bounding-box plus font-size analysis to recover superscripts and subscripts as plain-text tokens (**x^2**, **H_2O**). Without this recovery, **H■O** silently flattens to **H2O** and breaks both embedding similarity and quiz correctness.
- PyPDF2 (secondary) when PyMuPDF is unavailable; lossier on formatting but permissive on malformed PDFs.
- Tesseract OCR (tertiary) on any page yielding under fifty characters of extracted text. I rasterise at 200 DPI — high enough for clean print scans, low enough to keep ingestion within seconds.

After extraction, every line passes a normaliser that unifies non-breaking spaces, the three dash codepoints, smart quotes, and whitespace-before-punctuation. This is load-bearing: without it, the embedder treats **photo-synthesis** (with a soft hyphen) and **photosynthesis** as different tokens, which silently degrades recall.

Per page I extract structural metadata via regex: section codes (**2.1.3**, **Exercise 1.5**), section titles, the parent-section breadcrumb, content type (text / exercise / example). This metadata feeds the reranker and the citation layer.

3.2 Chunking — Paragraph-First, Heading-Aware

I deliberately did not use LangChain's **RecursiveCharacterTextSplitter**. Recursive character splitting is structurally blind; it cheerfully cuts across section boundaries and corrupts retrieval. My chunker accumulates paragraphs (double-newline split) up to a 750-token budget with 100 tokens of overlap, and adds two structural rules on top:

- A heading match (section-heading or exercise-heading regex on a paragraph's first line) forces a chunk boundary even at low fill, preserving the document's pedagogical units.
- Token counting uses **tiktoken** with the **gpt-3.5-turbo** encoding for stability across providers. If tiktoken fails to import, the chunker falls back to a **len(text) // 4** estimate, which slightly over-counts and therefore never exceeds the embedding window.

I chose 750 tokens because it comfortably fits inside the embedding model's 256-token effective window after the model's internal pooling, yet is large enough that a chunk typically contains a complete worked example. The 100-token overlap keeps cross-chunk references (a definition introduced in one chunk and applied in the next) recoverable through either neighbour.

Each chunk carries: **chunk_id**, **text**, **token_count**, and a metadata block with **page_number**, **chapter**, **section_code**, **section_title**, **topic**, **headers** (breadcrumb), and **content_type**.

I also precompute a single document-wide **study_context** of at most 8000 characters, sampled from the first twelve sections plus representative chunks from the beginning, middle, and end. Summary and quiz generation prefer this distilled context over the raw text — this single optimisation cut summary latency on Ollama from roughly 25 s to roughly 6 s on a 300-page textbook.

3.3 Embeddings — all-MiniLM-L6-v2 with Query-Side Batching

The default embedder is **sentence-transformers/all-MiniLM-L6-v2** (384 dimensions, L2-normalised). Ollama's **nomic-embed-text** is supported as an alternative for deployments that want a single inference stack.

I picked MiniLM-L6 for three reasons: 384 dimensions keep ChromaDB lookups sub-millisecond on CPU; the model is ~80 MB on disk so it can ship with the project without a download phase; and on the informal evaluation set I ran during development it outperformed **bge-small** and **e5-small-v2** on the kind of imprecise, structurally-referenced queries students actually ask.

Encoding runs through **asyncio.to_thread** so the synchronous encoder does not block the FastAPI event loop. I always pass **normalize_embeddings=True**, which reduces cosine similarity to a dot product at query time and gives rerank scores a comparable scale.

When the Q&A pipeline rewrites a question into up to four retrieval variants, all four variants are embedded in a single batched **encode()** call — roughly 250 ms saved per request on CPU.

4. Vector Database

4.1 Why ChromaDB

I evaluated Pinecone, Qdrant, Weaviate, and pgvector. School LLM has to run on a single mid-range machine in a school IT closet, survive a power cycle without re-indexing, and be inspectable by a non-infrastructure-engineer. ChromaDB's **PersistentClient** with on-disk storage satisfied all three; the managed cloud stores added an operational dependency the deployment profile does not justify.

4.2 One Collection Per PDF

The community is split between a single shared collection with a metadata filter for the PDF, versus one collection per PDF. I chose collection-per-PDF, named **pdf_<md5(schema_version:url)>**.

Rationale: HNSW query latency on a shared collection grows with library size, while per-PDF collections cap each query's search space to the document in active study. The MD5 includes a schema version (**2026-03-31-section-chunks-v1**) so that re-chunking the corpus under a new strategy produces a fresh namespace without conflicting with the old vectors. The HNSW index is configured with **hnsw:space=cosine** to match the L2-normalised embeddings.

4.3 Idempotent Indexing

Before inserting documents I check **collection.count() > 0** and skip re-embedding when the PDF is already indexed. In a classroom where the same PDF is opened by twenty students in a day, this turns the first 40-second ingestion into a no-op for every subsequent access.

4.4 Two-Stage Retrieval with a Metadata-Aware Reranker

Vanilla "embed-and-top-k" is mediocre for school traffic because students phrase queries imprecisely (**tell me about chapter 3**), reference structure (**explain exercise 2.1.5**), and ask follow-ups whose context lives in conversation history. I therefore implemented two-stage retrieval in **backend/vector_db.py**.

Stage one retrieves three times the requested count by cosine distance. Stage two reranks using **base_score = 1 - distance** plus the following additive boosts, capped at 1.0:

- +0.10 per query term appearing verbatim in the chunk, capped at +0.30.
- +0.15 if the query string appears in the chunk's **topic** metadata.
- +0.25 if a section code in the query exactly matches the chunk's **section_code**.
- +0.35 if the chunk's **section_code** is in the caller's **preferred_section_codes** list, and +0.20 for a parent / child of a preferred section.
- +0.15 if any query term appears in the **section_title**.

The boost weights were tuned on the test set I built from real student questions during development. Pure semantic matches still win when the question is open-ended; structural questions now correctly bias toward the right chunks even when raw cosine misranks them.

I chose this over a cross-encoder reranker (**bge-reranker**, **cohere-rerank**) because a cross-encoder doubles per-query CPU cost. The metadata-aware reranker runs in microseconds

and exploits structural signals a cross-encoder cannot see.

5. Prompt Engineering

5.1 Q&A — Rule-Based System Prompt with Intent and Audience Layers

The Q&A system prompt in **backend/ai/qa.py** opens with a role line, then seven explicit prohibitions. The rules are phrased as prohibitions rather than positive instructions because smaller models (qwen2.5:3b especially) copy structural cues from the user message into the answer unless explicitly forbidden. Key rules:

- Never repeat source passages verbatim — paraphrase.
- Never use the words **Question**, **Evidence**, **Source**, **Task**, **PDF**, **Context Hint** as headings.
- Never copy the prompt structure into the answer.
- Use only facts from the provided passages.
- For out-of-document questions, reply with one fixed sentence. A model-phrased refusal frequently leaks ungrounded knowledge (**I can't answer from the PDF, but generally...**); a fixed sentence eliminates that failure mode.
- For math, plain text only (**x^2**, **sqrt(x)**) — Streamlit does not render LaTeX without a MathJax dependency.

The system prompt is then augmented at request time with two dynamic blocks. An **intent classifier** inspects the question and the matched sections and returns one of: **direct_qa**, **problem_solving**, **section_explanation**, **follow_up**, **summary**, **chapter_list**, **overview**. Each intent injects its own structural prescription — problem-solving gets numbered solution steps and a bolded final-answer line; section explanation gets a definition-first opening; chapter listing gets a numbered list with one-sentence descriptions. An **audience instruction** is added based on role: administrators get a 3–5 bullet style with a 350-token budget; students get a 5–9 sentence narrative with a 700-token budget. Token budget is then further bumped per intent (+150 for problem-solving, +80 for explanation / summary).

Context construction: top four reranked chunks, 700 characters per chunk, 2800 characters total. Beyond ~3000 characters answer quality on small models degrades sharply; between 2000 and 3000 it is essentially flat. Each chunk is prefixed with a citation tag like **[S1|p5|2.3.1]** for in-answer references.

Conversation history is included but capped at the last four turns. Beyond four turns, context drift outweighs the benefit.

5.2 Grounding Check — Refusal Before Inference

Before any LLM call, a grounding check runs over the retrieval scores. It returns true (refuse) unless the combination of max rerank score, average score, question-term match count, coverage ratio, and best-per-chunk overlap clears thresholds tuned per intent. The thresholds are layered: chapter-specific questions accept at max ≥ 0.45, explicit section-code matches accept at 0.35, baseline questions require max ≥ 0.45 plus term-match floors that scale with question length.

When the check fails, the fixed refusal sentence is returned with zero LLM calls. This saves inference per refused question and — more importantly — prevents the model from being seduced into hallucinating on a weakly matched context.

5.3 Summary — Structurally Adaptive

A naive summary prompt ("summarise the following") works on a single-topic article and fails on a multi-chapter textbook (over-focus on chapter one) or a worksheet (it tries to summarise individual exercises). My system prompt in **backend/ai/summary.py** instead instructs the model to *first decide* the document type and then choose a structure:

- Multi-chapter textbook → bullet list, one sentence per chapter, naming each chapter.
- Single-topic explainer → 2–3 short paragraphs covering key ideas.
- Notes / mixed / exam paper → 2–3 paragraphs organised by theme.

The branching happens inside the model because the model can read the text and judge structure far more reliably than any external heuristic.

Two hard rules sit at the bottom: preserve every chapter name, formula, technical term, and proper noun verbatim (so a student searching for **cellular respiration** still finds it); and always end on a complete sentence — if approaching the token limit, finish at the next sentence boundary rather than starting a thought you cannot complete. A defensive **_trim_to_last_sentence** post-processor walks back to the last terminator if the model ignores the instruction.

The bundled call generates short and detailed summaries in a single JSON-mode request, with a three-stage defensive parser (direct **json.loads**, then field-level regex, then loose key-value parse) so malformed output from small models is still recoverable.

5.4 Quiz — Schema-First, with Type Mandates and a Five-Stage Parser

Quiz output must be machine-consumable JSON with a strict schema. The system prompt in **backend/ai/quiz.py** opens with **Return ONLY a valid JSON object: {"questions": [...]}**, then enumerates every field, then provides type-specific schemas:

- **mcq** — **options** dict with keys A/B/C/D; **correct_answer** is the letter.
- **true-false** — **options = {"A": "True", "B": "False"}**; **correct_answer** is literally **True** or **False**.
- **fill-in-blank** — question must contain _____; options empty; **correct_answer** is the missing word or phrase.
- **short-answer** — empty options; **correct_answer** is a 1–2 sentence model answer; additionally requires a **keywords** field (3–5 crucial terms) that the auto-grader uses as a rubric.
- **long-answer** — empty options; **correct_answer** is a 3–6 sentence model answer; **keywords** = 5–8 crucial terms.

The keyword-rubric extension on free-text answers is non-obvious but critical: grading free text without a rubric is unreliable, and asking the *generation* model to emit the rubric alongside the answer is far cheaper than running a second LLM call.

When the caller requests a single question type, I append a *mandate clause* — for example, for MCQ: **MANDATORY: every question MUST be MCQ ... A question without four options is**

INVALID and will be REJECTED. Without this, qwen2.5:3b silently downgrades MCQ to short-answer (because short-answer is easier to generate). A post-generation validator (**_is_valid_for_type**) drops any non-conforming question before it reaches the student.

The output goes through a five-stage parsing pipeline: direct **json.loads**; fenced-code-block extraction; regex for the **{...}** block containing **questions**; flat-array regex; plain-text fallback that recovers questions from numbered lines. Each stage logs why it had to engage — invaluable when a model upgrade silently changes output formatting.

When called with a search query and a PDF identifier, the quiz module performs RAG: it retrieves eight chunks (850 chars each, 5200 chars total) and uses those instead of the full document, giving topic-focused quizzes.

6. Multi-Provider LLM Strategy

6.1 The Constraint

Anthropic Claude gives the best output quality and instruction-following but costs per token, requires reliable internet, and has rate limits that bite when thirty students hit the API simultaneously. Local Ollama with qwen2.5:3b is free, offline-capable, and unrestricted, but answer quality is lower and a CPU-only machine cannot serve many concurrent students. Neither alone is acceptable for school deployment.

6.2 Protocol Abstraction

In **backend/ai/llm_client.py** I defined an **LLMClient** Protocol with a single **chat()** method (OpenAI-style messages-in, string-out). Concrete implementations: **OllamaWrappedClient**, **AnthropicLLMClient**, **OpenRouterLLMClient**. The caller never sees provider-specific shape — system message extraction, cache-control attachment, and provider-specific HTTP all live inside the implementation.

6.3 Multi-Tier Routing within Anthropic

Each client exposes three model properties: **generation_model**, **evaluation_model**, **naming_model**. For Anthropic these default to:

- **generation_model = claude-sonnet-4-6** — student-facing Q&A, summaries, quiz generation. Quality differential over Haiku matters here; latency premium (1–2 seconds) is acceptable.
- **evaluation_model = claude-haiku-4-5-20251001** — the LLM-as-judge pipeline that scores hundreds of artefacts per batch. Per-call quality matters less than throughput and cost; Haiku is roughly four times cheaper per token than Sonnet, and on the evaluation rubrics its scores correlate well with human judgement.
- **naming_model = claude-haiku-4-5-20251001** — auto-generating chat-session titles. Low-stakes utility task, Haiku is sufficient.

A single environment variable does not force every call onto one model — the architecture gets Sonnet quality where it matters and Haiku economics where it does not.

6.4 Prompt Caching

The Anthropic client implements ephemeral prompt caching for the static system prompt. The system message is wrapped in a content block with **cache_control = {"type": "ephemeral"}** and the request carries the **anthropic-beta: prompt-caching-2024-07-31** header. The ephemeral cache window (~5 minutes) matches the duration of a typical evaluation batch — the long static judge rubric (~400 tokens) is paid for once and then read from cache for every subsequent item. Empirically: 25–50% reduction in evaluation token spend depending on batch size, plus a smaller but measurable latency drop because the cache hit shortcuts input processing.

6.5 Fallback with Cooldown

The **FallbackLLMClient** wraps a primary and a secondary client and maintains a per-provider cooldown timestamp. On primary failure, the cooldown is set to **now + 60s** and the next requests skip the dead provider entirely. The cooldown matters because the failure mode I protect against is sustained outage (rate-limit on our IP, regional incident, expired key) — without it, every user request pays a multi-second timeout to the dead provider before retrying. The cooldown is symmetric on the secondary, so when both providers fail the system fails fast rather than chasing both.

6.6 The OpenRouter Experiment (Rolled Back)

I briefly routed evaluation through OpenRouter's free Nemotron 30B model, which would have made evaluation effectively free. End-to-end testing showed the free tier caps at 50 requests per day and 16 per minute — both too tight for a single school-day's traffic. I rolled the integration back. The code is preserved (**OpenRouterLLMClient**) for the future possibility of paid credits, but the active evaluation path is back on Anthropic Haiku. Walking away from working code was the right call; over-claiming "zero-cost evaluation" would have been a latent footgun in production.

7. Performance and Latency Engineering

The latency target was 3 seconds for Q&A and 5 seconds for summaries on the Anthropic-backed path; the Ollama path is allowed up to 12 seconds because the constraint is CPU inference, not network. The levers:

- **End-to-end async.** Every AI call is **async**. The synchronous sentence-transformer encoder runs through **asyncio.to_thread**. The Anthropic SDK and HTTP paths are both thread-pooled. One FastAPI process serves multiple concurrent Q&A requests without queuing.
- **Idempotent indexing.** Skip re-embedding when the ChromaDB collection is non-empty — turns a 40-second indexing operation into a no-op on repeat uploads.
- **Query-side embedding batching.** Up to four Q&A retrieval variants are embedded in a single **encode()** call — ~250 ms saved per request on CPU.
- **Prompt caching** (Section 6.4) — 25–50% token-spend reduction on batched evaluations.
- **Study-context precomputation** (Section 3.2) — summary call on a 300-page textbook ingests an 8000-character distillation, not the raw text. Reduced Ollama summary time from ~25 s to ~6 s.

- **Per-minute burst middleware.** A sliding-window rate limiter (60 general req/min, 20 AI req/min) sits in front of every endpoint, keyed by JWT email or IP. Anti-burst protection, not quota management — exists to stop runaway scripts.
- **Explicit timeouts.** PDF download 30 s; Ollama chat 60 s; Anthropic HTTP 120 s; OpenRouter HTTP 120 s. These feed the fallback cooldown — exceeding a timeout engages the wrapper.

8. Self-Evaluating Quality Assurance

A generative system without continuous quality control is irresponsible. **backend/evaluation/judge.py** defines three independent LLM-as-judges, each emitting a single JSON object on a 0.0–1.0 scale per metric plus a one-sentence rationale:

- **Reference-free Q&A judge** scores every student Q&A pair on **faithfulness** (consistent with retrieved chunks?) and **answer_relevance** (actually addresses the question?). It is reference-free because in a real RAG system there is no ground-truth answer to compare against.
- **Summary judge** scores generated summaries on **faithfulness**, **completeness**, and **semantic_match** against the source.
- **Quiz judge** scores each generated quiz question on **validity** (well-formed?), **correctness** (is the stated answer right?), and **faithfulness** (grounded in the supplied context?).

Every rubric explicitly instructs the judge to award **partial credit** in the 0.4–0.9 band rather than binary 0 / 1. Binary scores hide whether the system is incrementally improving and amplify individual judging mistakes.

Source-context budgets per judge: 3000 chars for Q&A and quiz, 4000 chars for summary; the runner allows source excerpts up to 15000 chars before truncation so the summary judge sees enough of the document to verify completeness.

There is no golden set. I built one early and removed it. Real student questions diverge significantly from any pre-curated golden set, and a system that scores 95% on its own golden set and 60% on real traffic is worse than useless — it manufactures false confidence. The current pipeline scores real interactions in batches that an administrator launches from the dashboard; the resulting scores feed back into the prompt-engineering decisions in Section 5.

9. Operational Controls

The role-permissions matrix (a MongoDB document, editable from the admin dashboard) lets administrators toggle individual AI features per role without redeployment. The audit log captures every authenticated action with user, role, route, status, and redacted payload. The in-progress operational sprint adds admin-controlled per-day rate limits — a singleton settings doc + a **(user_id, feature, day)** counter collection + a FastAPI dependency that returns HTTP 429 with a midnight-reset timestamp when the limit is exceeded. Administrators are exempt at the server level so configuration mistakes cannot lock them out.

10. Conclusion

The architecture rests on four pillars: a structurally aware RAG pipeline that respects how textbook authors organise material; a multi-provider LLM strategy that mixes Claude quality with local Ollama resilience and economy; module-specific prompt engineering calibrated against the actual failure modes of the underlying models; and a continuous self-evaluation loop that scores real production traffic on partial-credit rubrics. Every parameter cited in this document is grounded in code under **backend/** and can be reproduced by reading the repository.

Prepared by Gangadhar Reddy — 26 May 2026.